

TP15 — Automatiser une action via API DNA Center

Formation NetOps : Automatiser la gestion de son Infrastructure Réseau — Ambient IT

Description

Cisco DNA Center expose une API REST avec un cycle bien particulier : authentification par identifiants classiques (renvoie un token), interrogation de l'inventaire des équipements, puis déclenchement d'actions qui sont asynchrones — l'appel renvoie immédiatement un identifiant de tâche, qu'il faut ensuite interroger ("polling") jusqu'à ce qu'elle soit terminée. Ce TP simule ce cycle avec un mock DNA Center, et automatise le déploiement d'un template de configuration (VLAN invité) sur tous les sites NexaCorp.

Prérequis

- TP1 à TP14 validés

Étapes

Étape 1 — Créer sa branche

```
cd netops-project-<nom-groupe>
git checkout dev
git pull origin dev
git checkout -b <prenom>-<nom>-jour4-tp15
```

Étape 2 — Créer le mock DNA Center

```
cat > scripts/mock_dnac_api.py << 'EOF'
import time
import uuid

from flask import Flask, request, jsonify

app = Flask(__name__)

VALID_USER = "admin"
VALID_PASS = "dnacpass123"
VALID_TOKEN = "dnac-fake-token-xyz"
```

```

DEVICES = [
    {"id": "dev-siege", "hostname": "siege-router", "managementIpAddress": "10.10.10.1", "reachabilityStatus": "Reachable"},
    {"id": "dev-agence1", "hostname": "agence1-router", "managementIpAddress": "10.10.10.2", "reachabilityStatus": "Reachable"},
    {"id": "dev-agence2", "hostname": "agence2-router", "managementIpAddress": "10.10.10.3", "reachabilityStatus": "Reachable"},
]

TASKS = {}

def check_auth_token():
    return request.headers.get("X-Auth-Token") == VALID_TOKEN

@app.route("/dna/system/api/v1/auth/token", methods=["POST"])
def auth_token():
    auth = request.authorization
    if not auth or auth.username != VALID_USER or auth.password != VALID_PASS:
        return jsonify({"error": "Unauthorized"}), 401
    return jsonify({"Token": VALID_TOKEN})

@app.route("/dna/intent/api/v1/network-device", methods=["GET"])
def network_device():
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    return jsonify({"response": DEVICES})

@app.route("/dna/intent/api/v1/template-programmer/template/deploy", methods=["POST"])
def deploy_template():
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    body = request.get_json(force=True)
    if not body or "templateId" not in body or "targetInfo" not in body:
        return jsonify({"error": "templateId et targetInfo sont requis"}), 400
    task_id = str(uuid.uuid4())[:8]
    TASKS[task_id] = {"polls": 0}
    return jsonify({"deploymentId": f"dep-{task_id}", "taskId": task_id})

@app.route("/dna/intent/api/v1/task/<task_id>", methods=["GET"])
def get_task(task_id):
    if not check_auth_token():
        return jsonify({"error": "Invalid or missing X-Auth-Token"}), 401
    if task_id not in TASKS:
        return jsonify({"response": {"progress": "Unknown task", "isError": True}}), 404
    TASKS[task_id]["polls"] += 1
    if TASKS[task_id]["polls"] < 3:

```

```

        return jsonify({"response": {"progress": "IN_PROGRESS", "isError": False, "endTime": None}})
    return jsonify({"response": {"progress": "TEMPLATE_DEPLOYED", "isError": False, "endTime": int(time.time() * 1000)}})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=6000, debug=False)
EOF

```

DNA Center distingue volontairement l'authentification (identifiants classiques, une seule fois) du token qui en résulte (utilisé ensuite pour chaque appel) — d'où deux mécanismes différents dans ce mock : Basic Auth pour /auth/token, puis X-Auth-Token pour tout le reste.

Étape 3 — Créer le client Python

```

cat > scripts/netops_client/dnac_client.py << 'EOF'
import time

import requests

class DNACError(Exception):
    """Erreur levee lors d'un echange avec l'API DNA Center."""
    pass

class DNACClient:
    """Client minimal pour l'API Cisco DNA Center (authentification, inventaire, deploiement de template)."""

    def __init__(self, base_url, username, password, timeout=5):
        self.base_url = base_url.rstrip("/")
        self.username = username
        self.password = password
        self.timeout = timeout
        self.token = None

    def authenticate(self):
        url = f"{self.base_url}/dna/system/api/v1/auth/token"
        response = requests.post(url, auth=(self.username, self.password), timeout=self.timeout)
        if response.status_code != 200:
            raise DNACError(f"Authentification refusee ({response.status_code})")
        self.token = response.json()["Token"]
        return self.token

    def _headers(self):
        if not self.token:
            raise DNACError("Non authentifie : appelez authenticate() d'abord")
        return {"X-Auth-Token": self.token}

```

```

def get_devices(self):
    url = f"{self.base_url}/dna/intent/api/v1/network-device"
    response = requests.get(url, headers=self._headers(), timeout=self.timeout)
    if response.status_code != 200:
        raise DNACError(f"Erreur inventaire ({response.status_code}) : {response.text}")
    return response.json()["response"]

def deploy_template(self, template_id, device_id, params=None):
    url = f"{self.base_url}/dna/intent/api/v1/template-programmer/template/deploy"
    body = {
        "templateId": template_id,
        "targetInfo": [{"id": device_id, "type": "MANAGED_DEVICE_UUID", "params": params or {}}],
    }
    response = requests.post(url, headers=self._headers(), json=body, timeout=self.timeout)
    if response.status_code != 200:
        raise DNACError(f"Echec du declenchement du deploiement : {response.text}")
    return response.json()["taskId"]

def wait_for_task(self, task_id, max_attempts=10, delay=1):
    url = f"{self.base_url}/dna/intent/api/v1/task/{task_id}"
    for attempt in range(1, max_attempts + 1):
        response = requests.get(url, headers=self._headers(), timeout=self.timeout)
        if response.status_code != 200:
            raise DNACError(f"Erreur lors du suivi de la tache : {response.text}")
        data = response.json()["response"]
        if data.get("isError"):
            raise DNACError(f"Tache en erreur : {data}")
        if data["progress"] != "IN_PROGRESS":
            return data
        time.sleep(delay)
    raise DNACError(f"Tache {task_id} toujours en cours apres {max_attempts} tentatives")
EOF

```

Étape 4 — Exposer le client dans le package

```

cat > scripts/netops_client/__init__.py << 'EOF'
from .dnac_client import DNACClient, DNACError

__all__ = ["DNACClient", "DNACError"]
EOF

```

Étape 5 — Créer le script d'automatisation

```

cat > scripts/automate_dnac_action.py << 'EOF'
import os
import sys

```

```

from netops_client import DNACClient, DNACError

DNAC_URL = os.environ.get("DNAC_URL", "http://localhost:6000")
DNAC_USER = os.environ.get("DNAC_USER", "admin")
DNAC_PASS = os.environ.get("DNAC_PASS")

if not DNAC_PASS:
    print("ERREUR : variable d'environnement DNAC_PASS non definie.")
    sys.exit(1)

TEMPLATE_ID = "tpl-vlan-invite-v1"

def main():
    client = DNACClient(base_url=DNAC_URL, username=DNAC_USER, password=DNAC_PASS)

    print(f"Authentification sur {DNAC_URL}...")
    client.authenticate()
    print("Authentification reussie.\n")

    devices = client.get_devices()
    print(f"{len(devices)} equipement(s) trouve(s) dans l'inventaire DNA Center :")
    for d in devices:
        print(f"  - {d['hostname']} ({d['managementIpAddress']}) : {d['reachabilityStatus']}")
    print()

    for device in devices:
        print(f"Deploiement du template '{TEMPLATE_ID}' sur {device['hostname']}...")
        task_id = client.deploy_template(
            template_id=TEMPLATE_ID,
            device_id=device["id"],
            params={"vlan_id": 99, "vlan_name": "GUEST"},
        )
        print(f"  Tache creee : {task_id}, attente de la fin du deploiement...")
        try:
            result = client.wait_for_task(task_id, max_attempts=6, delay=1)
            print(f"    -> {result['progress']}\n")
        except DNACError as e:
            print(f"    -> ECHEC : {e}\n")

if __name__ == "__main__":
    try:
        main()
    except DNACError as e:
        print(f"Erreur DNA Center : {e}")
        sys.exit(1)

```

EOF

Étape 6 — Lancer le mock et tester l'automatisation

```
nohup python scripts/mock_dnac_api.py > /tmp/dnac.log 2>&1 &  
export DNAC_PASS=dnacpass123  
python scripts/automate_dnac_action.py
```

Résultat attendu :

Authentication sur http://localhost:6000...
Authentication reussie.

3 équipement(s) trouve(s) dans l'inventaire DNA Center :

- siege-router (10.10.10.1) : Reachable
- agence1-router (10.10.10.2) : Reachable
- agence2-router (10.10.10.3) : Reachable

Deploiement du template 'tpl-vlan-invite-v1' sur siege-router...

Tache creee : 556fc0c0, attente de la fin du deploiement...

-> TEMPLATE_DEPLOYED

(et ainsi de suite pour les deux autres sites)

Étape 7 — Tester le rejet d'un mauvais mot de passe

```
export DNAC_PASS=mauvais_mot_de_passe  
python scripts/automate_dnac_action.py
```

Résultat attendu : Erreur DNA Center : Authentication refusee (401), arrêt propre (code de sortie 1), sans tenter les étapes suivantes.

```
export DNAC_PASS=dnacpass123
```

Étape 8 — Committer et pousser

```
git add scripts/mock_dnac_api.py scripts/netops_client/dnac_client.py scripts/netops_client/__init__.py scripts/automate_dnac_action.py  
git commit -m "TP15 : automatisation d'une action DNA Center (auth, inventaire, deploiement asynchrone)"  
git push -u origin <prenom>-<nom>-jour4-tp15
```

Ouvrir une pull request <prenom>-<nom>-jour4-tp15 → dev.

Comprendre le TP

Le cycle d'authentification DNA Center est en deux temps, volontairement séparés : un appel initial avec de vrais identifiants pour obtenir un token, puis ce token seul pour toutes les requêtes suivantes — le mot de passe ne circule qu'une fois, jamais répété à chaque appel. C'est un modèle qu'on retrouve dans la plupart des grandes APIs réseau (ACI, SD-WAN inclus).

Le déclenchement asynchrone est la caractéristique la plus spécifique de ce type d'API : appeler `deploy_template` ne dit pas "c'est fait", ça dit "j'ai bien reçu la demande, voici un identifiant pour suivre son avancement" (`taskId`). `wait_for_task` interroge

ensuite régulièrement cet identifiant jusqu'à obtenir un statut final — le même principe que le `until/retries` vu en Ansible au TP9, mais écrit à la main en Python, avec `time.sleep()` entre chaque tentative.

Cette conception a une raison concrète : déployer une configuration sur un vrai équipement réseau peut prendre plusieurs secondes à plusieurs minutes — une API synchrone obligerait le client à rester bloqué en attente pendant tout ce temps. Le modèle asynchrone découple “demander l'action” de “attendre le résultat”.

Validation

- ☐ `python scripts/automate_dnac_action.py` s'authentifie, liste 3 équipements, et déploie le template sur chacun avec succès
- ☐ Chaque déploiement passe par un statut `IN_PROGRESS` avant `TEMPLATE_DEPLOYED` (le polling fonctionne réellement)
- ☐ Un mauvais mot de passe fait échouer le script proprement dès l'authentification, sans tenter la suite
- ☐ Une pull request `<prenom>-<nom>-jour4-tp15` → dev est ouverte